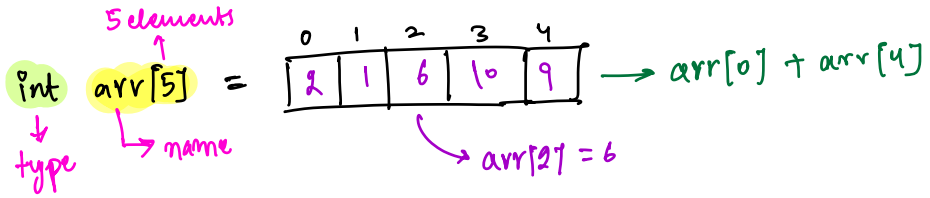
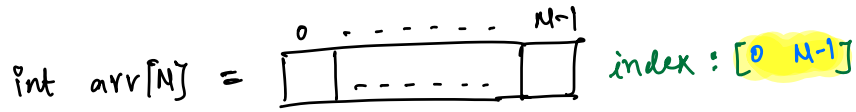
Till now

- T.C
- Iterations
- BigO
- S.C



Note:- Accessing arr[i] from arr[] is always O(1)



// Print arr[]

```
for(int i=0; i<N; i++){
|   print(arr[i]);
}
```

O(1)

T.C to print N elements = O(N)

1) To access an ith element from an arr[] is constant time → O(1)
2) How to iterate an array.

Ques Given N array elements, count no. of elements having atleast 1 element, greater than itself

Note:- No external libraries.

e.g. arr[7] = {-3, -2, 6, 8, 4, 8, 5}

ans=5, max 8, max-cnt = 2

ans = N - max-cnt

= 7 - 2

= 5

e.g. arr[8] = {2, 3, 10, 7, 3, 2, 10, 8}

ans=6, max=10, max-cnt = 2

ans = 8 - 2

= 6

e.g. arr[10] = {2, 5, 1, 4, 8, 0, 8, 1, 3, 8}

ans=7, max=8, max-cnt=3

ans = 10 - 3 = 7

Observations:-

1) For every array, we can not have anything greater than the max element.

2) Get count of the max ele in arr[] = cnt

3) ans = N - cnt

Pseudo code

1) iterate and get max → arr[0] / MIN_VALUE

```
int max_val = 0 ;
for(i=0;i<N;i++){
    if(max_val < arr[i]){
        max_val = arr[i];
    }
}
```

→ N iterations

2) iterate and get count of max = cnt

```
int cnt=0
for(i=0;i<N;i++){
    if(arr[i] == max_val){
        cnt++;
    }
}
```

→ N iterations

3) return N - cnt; → 1 iteration

T.C = O(N+N+1) ⇒ O(2N+1) ⇒ O(N)

TODO: Try and do it in a single loop.

S.C = O(1)

Ques 2 Given N array elements, check if (→ boolean) there exists a pair i, j s.t. arr[i] + arr[j] == K and i != j

Note:- i and j are index values, K is given sum
↳ input : arr[], N, K

eg. arr[] = 3 -2 1 4 3 6 8 (indices 0 1 2 3 4 5 6), N = 7

K = 10 : i j
3 5 → arr[3] + arr[5] = 10 → return true

eg. arr[] = 2 4 -3 7 (indices 0 1 2 3)

K = 5 : i j → return false
X X

eg. arr[] = 2 4 -3 7 (indices 0 1 2 3)

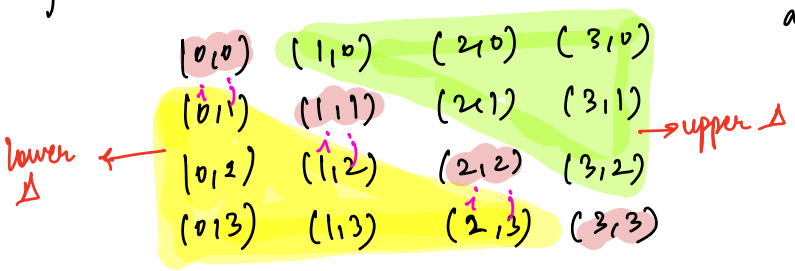
K = 8 : i j
1 1 → arr[i] + arr[i] = 8 → return false
i should not be equal to j

Sol 1:- Check all pairs

e.g. N = 4 $\{0, 1, 2, 3\}$

arr[i] + arr[j] == arr[j] + arr[i]

arr[0] + arr[1] == arr[1] + arr[0]




```
for(i=0; i<N; i++){
    for(j=0; j<N; j++){
        if(i==j) continue;        → we donot want i==j pairs.
        if(arr[i] + arr[j] == K) {
            return true;
        }
    }
}
return false;
```

T.C = $O(N^2)$

Optimisation

```
for(i=0; i<N; i++){
    for(j=i+1; j<N; j++){
        if(arr[i] + arr[j] == K) {
            return true;
        }
    }
}
return false;
```

i	j	# iterations
0	[1, N-1]	N-1
1	[2, N-1]	N-2
2	[3, N-1]	N-3
⋮	⋮	⋮
N-2	[N-1, N-1]	1
N-1	[N, X]	

total iterations = $1 + 2 + 3 + \dots + (N-3) + (N-2) + (N-1)$

sum of (N-1) natural numbers

Sum of N natural numbers ? $= \frac{(N)(N+1)}{2}$

substitute N-1

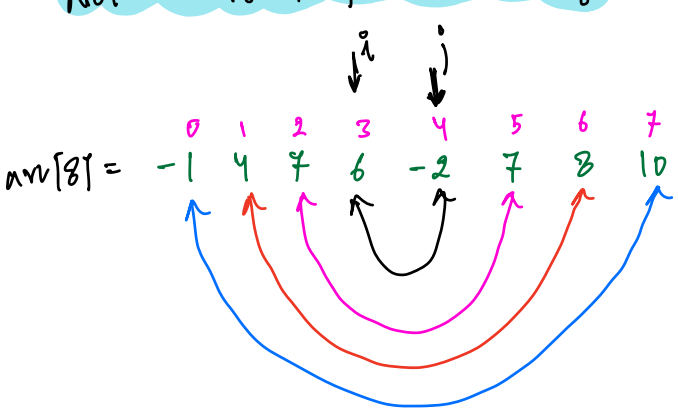
Sum of (N-1) natural numbers ? $= \frac{(N-1)(N-1+1)}{2} = \frac{(N)(N-1)}{2}$ → Better → slight optimi-sation

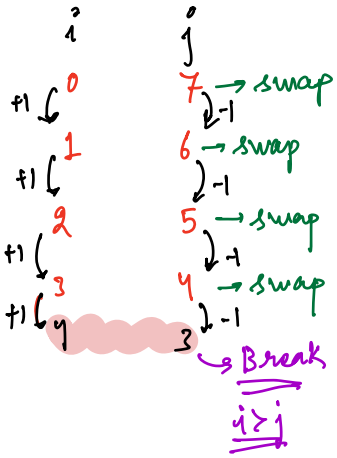
↓

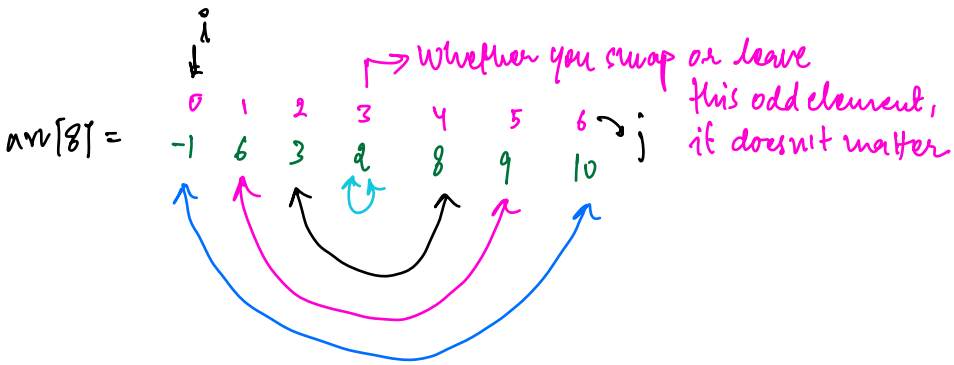
T.C = $O(N^2)$

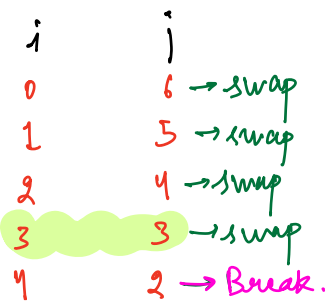
Ques3 Given an array, reverse an arr[] → expected S.C = O(1)

Note:- arr[] itself should change









Idea:- i=0; j=N-1

Keep on swapping until i<=j

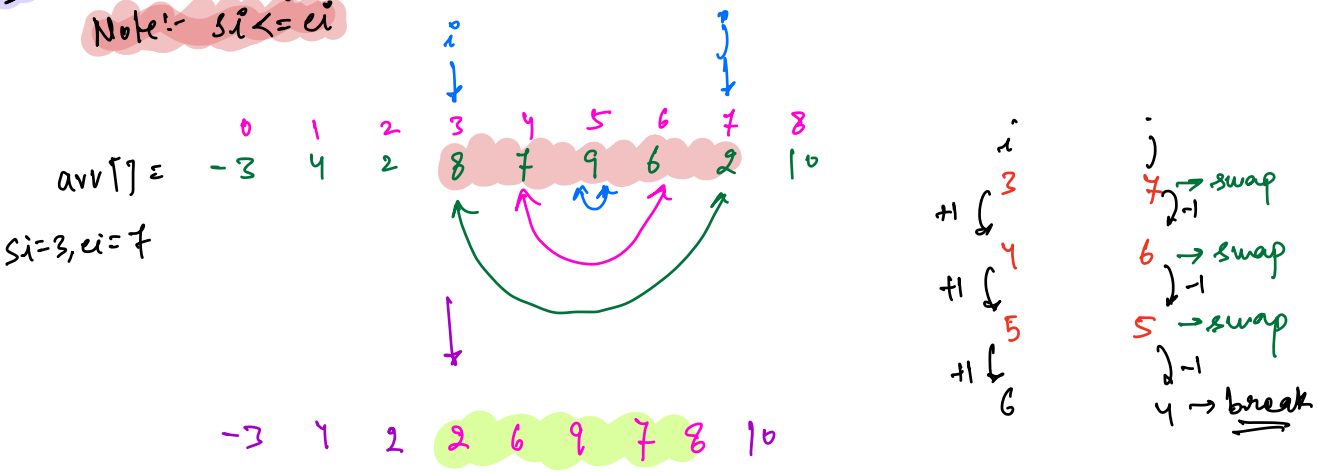
```
reverse(int[] arr, int N) {
    int i=0 ; j= N-1;        → i<j
    while ( i<=j ) {
        //swap arr[i] with arr[j]
        i++; j--;
    }
}
```

T.C = O(N)

S.C = O(1)

Ques Given N array elements and [si, ei] reverse array from [si - ei]

Note:- si <= ei




```
reverse(int[] arr, int N, int si, int ei)
    int i = si ; int j = ei;
    while (i <= j) {
        //swap arr[i] with arr[j]
        i++; j--;
    }
}
```

T.C (worst case) = O(N)

Ques Given N array elements, rotate array from last to first by K times.

Note:- Expected S.C = O(1)

	0	1	2	3	4	5	6
arr[7] =	3	-2	1	4	6	9	8
K=1 :	8	3	-2	1	4	6	9
K=2 :	9	8	3	-2	1	4	6
K=3 :-	6	9	8	3	-2	1	4

Quiz

A[] = 4, 1, 6, 9, 2, 14, 7, 8, 3

K = 4 :- 14 7 8 3 | 4 1 6 9 2

① last 4 elements become, first 4 elements.

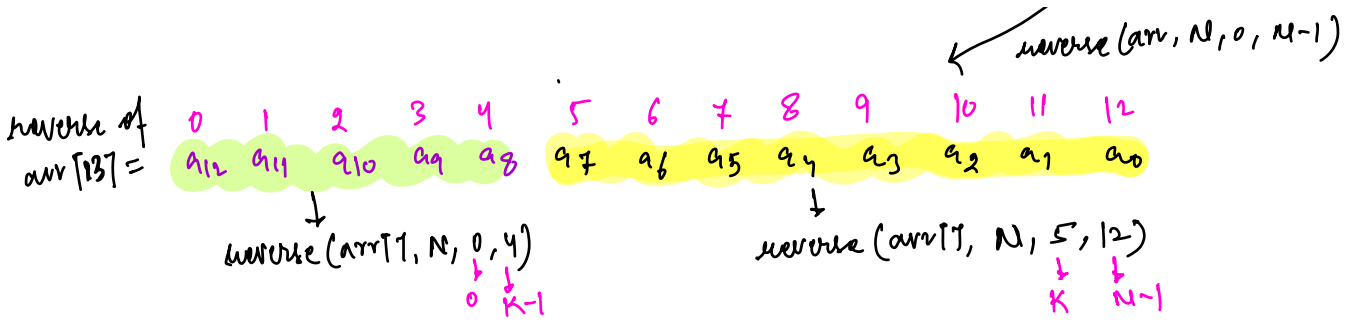
② First 5 elements become last 5 elements.

	0	1	2	3	4	5	6	7	8	9
arr[10] :-	-2	3	1	4	6	2	8	7	9	3
Rotate by k = 3 :-	7	9	3	-2	3	1	4	6	2	8

arr[13] = a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 a_8 a_9 a_{10} a_{11} a_{12}

Rotate by k=5 :- a_8 a_9 a_{10} a_{11} a_{12} a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7

Reversing comes as an intuition



Steps:-

① Reverse entire array → reverse(arr, N, 0, N-1) → N

② Reverse first K elements (0, K-1) → reverse(arr, N, 0, K-1) → K

③ Reverse last (N-K) elements. → reverse(arr, N, K, N-1) → N-K

total iterations = N + ~~K~~ + N - ~~K~~
= 2N

T.C = O(N)
S.C = O(1)

if K > N
let's say K = N+1

reverse(arr, N, 0, N-1) ✓ → 0, N
reverse(arr, N, 0, K-1)
reverse(arr, N, K, N-1)

i = 0, j = N
swap arr[i] with arr[j]
↓
arr[N] — ✗
↳ DANGER

arr[6] = a_0 a_1 a_2 a_3 a_4 a_5 → N=6

K=0 :- a_0 a_1 a_2 a_3 a_4 a_5

K=1 :- a_5 a_0 a_1 a_2 a_3 a_4

K=2 :- a_4 a_5 a_0 a_1 a_2 a_3

K=3 :- a_3 a_4 a_5 a_0 a_1 a_2

K=4 :- a_2 a_3 a_4 a_5 a_0 a_1

K=5 :- a_1 a_2 a_3 a_4 a_5 a_0

K=6 :- a_0 a_1 a_2 a_3 a_4 a_5

K			
6	12	18	→ %6 = 0
7	13	19	→ %6 = 1
8	14	20	→ %6 = 2
9	15	21	→ %6 = 3
10	16	22	→ %6 = 4
11	17	23	→ %6 = 5

[0, N-1]

if(K >= N) { K = K%N }

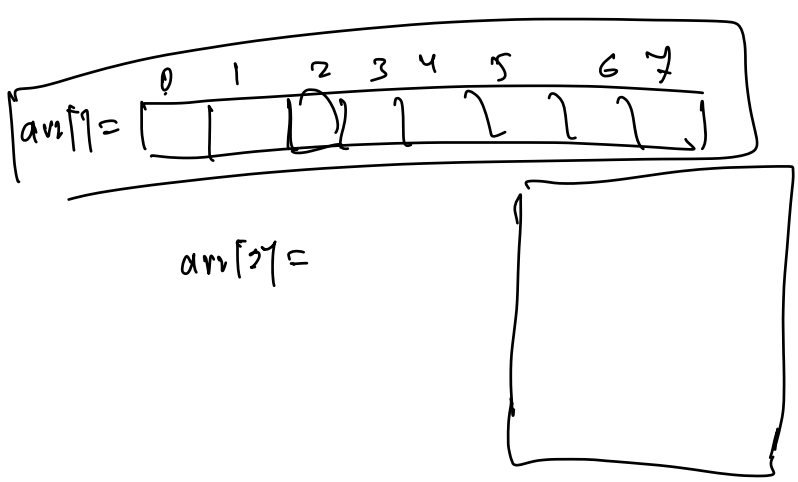
reverse(arr, N, 0, N-1)

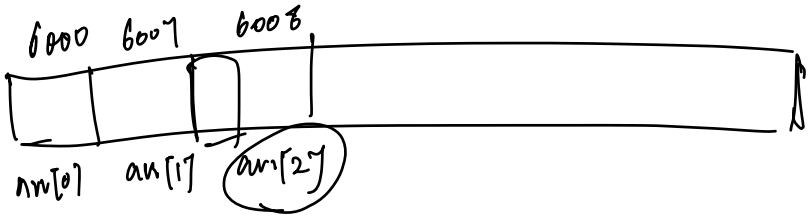
reverse(arr, N, 0, K-1)

reverse(arr, N, K, N-1)

TODO :- Try rotation from left to right.

1. Attending the class
2. Assignments.
3. Homework





arr[2] = arr[0] + ?

$\frac{N^2}{2} - N$

$\frac{N^2 - 2N}{2} = \frac{N(N-2)}{2}$